ออกแบบโปรเจกต์ All-in-One Productivity Hub

แนวทางนี้จะออกแบบให้เป็นเว็บรวมเครื่องมืออำนวยความสะดวกที่พัฒนาด้วย AI บน VS Code เก็บข้อมูลในอุปกรณ์เป็นหลัก และเผยแพร่ผ่าน GitHub Pages โดยเน้น ใช้งานง่าย ปรับแต่งหน้า Dashboard ได้ ทำงานออฟไลน์ได้ในส่วนที่รองรับ และไม่ผูกติดกับบริการที่ยังไม่ได้ตั้งค่า

ผมแนะนำให้ใช้แนวทาง Local-first + Static Web App + AI-assisted Development หมายถึงตัวเว็บทำงานในเบราว์เซอร์เป็นหลัก ไม่จำเป็นต้องมีเซิร์ฟเวอร์หรือฐานข้อมูลออนไลน์สำหรับฟีเจอร์พื้นฐาน

1. ภาพรวมระบบ

Productivity Hub

หน้า Dashboard ที่จัดวางและปรับแต่งได้

เวลาและปฏิทิน

นาฬิกา · จับเวลา · นับถอยหลัง

คำนวณและแปลงค่า

เครื่องคิดเลข · เปอร์เซ็นต์ · หน่วย

จัดการงาน

To-do · บันทึก · รายการโปรด

Developer Tools

JSON · Markdown · UUID · ข้อความ

Local Data Layer

LocalStorage · IndexedDB · Import / Export

GitHub Pages

เผยแพร่ไฟล์เว็บแบบ Static ผ่าน HTTPS

หลักการสำคัญ

- ไม่ต้องสมัครสมาชิกเพื่อใช้เครื่องมือพื้นฐาน
- ไม่ต้องตั้งค่า API Key หรือบริการภายนอกสำหรับฟีเจอร์หลัก
- ข้อมูลส่วนตัวเก็บในเบราว์เซอร์ของผู้ใช้เป็นหลัก
- ทุกฟีเจอร์ต้องมีสถานะโหลด ว่าง สำเร็จ และผิดพลาดอย่างเหมาะสม
- AI ที่ใช้พัฒนาเป็นผู้ช่วยเขียนโค้ด ไม่ใช่ส่วนบังคับในการใช้งานเว็บ
- การเปลี่ยนแปลงโค้ดต้องตรวจสอบและทดสอบก่อนนำขึ้นเว็บไซต์จริง

2. ข้อกำหนดระบบ (System Requirements)

กำหนดเอกสารหลักเป็น docs/REQUIREMENTS.md เพื่อให้ AI ทุกตัวอ้างอิงข้อกำหนดชุดเดียวกัน และไม่สร้างฟีเจอร์นอกขอบเขตโดยพลการ

2.1 ฟังก์ชันหลัก

รหัส	ฟังก์ชัน	ข้อกำหนดสำคัญ
FR-001	Dashboard	เพิ่ม ลบ ย้าย และจัดขนาดวิดเจ็ตได้
FR-002	Search	ค้นหาเครื่องมือจากชื่อและหมวดหมู่
FR-003	Clock	แสดงเวลาปัจจุบันตามอุปกรณ์
FR-004	Calculator	คำนวณพื้นฐาน เปอร์เซ็นต์ และลำดับการคำนวณได้ถูกต้อง
FR-005	Timer	จับเวลา นับถอยหลัง หยุด และเริ่มใหม่ได้
FR-006	Calendar	ดูปฏิทินและคำนวณวันระหว่างวันที่
FR-007	Tasks	เพิ่ม แก้ไข ลบ จัดลำดับ และทำเครื่องหมายว่างานเสร็จแล้ว
FR-008	Notes	จดบันทึก ค้นหา และบันทึกอัตโนมัติ

FR-009 Developer Tools	JSON, Markdown, UUID และเครื่องมือข้อความ
FR-010 Favorites	ปักหมุดเครื่องมือที่ใช้บ่อย
FR-011 Settings	ภาษา ธีม รูปแบบเวลา และการจัดวาง Dashboard
FR-012 Data Management	ส่งออก สำรอง และนำเข้าข้อมูล
FR-013 Offline	ใช้ฟังก์ชันที่รองรับได้โดยไม่ต้องเชื่อมต่ออินเทอร์เน็ต
FR-014 Accessibility	ใช้งานด้วยคีย์บอร์ดและรองรับการขยายหน้าจอ

2.2 ข้อกำหนดด้านคุณภาพ

- Responsive: ใช้งานได้บนหน้าจอมือถือ แท็บเล็ต และคอมพิวเตอร์
- Performance: หลีกเลี่ยงการโหลดเครื่องมือทุกตัวพร้อมกันหากไม่จำเป็น
- Reliability: ข้อมูลต้องไม่หายเมื่อเปลี่ยนหน้า หรือเกิดข้อผิดพลาดในวิดเจ็ตตัวใดตัวหนึ่ง
- Maintainability: แยกแต่ละเครื่องมือออกจากกัน ไม่เขียนทุกอย่างรวมในไฟล์เดียว
- Privacy: ข้อมูลผู้ใช้ไม่ถูกส่งออกนอกอุปกรณ์โดยไม่มีการดำเนินการที่ชัดเจนจากผู้ใช้
- Accessibility: ใช้แนวทาง WCAG 2.2 AA เป็นเป้าหมายในการออกแบบและทดสอบ
- Portability: สำรองข้อมูลเป็นไฟล์ที่ผู้ใช้สามารถเก็บไว้นอกเบราว์เซอร์ได้

2.3 ขอบเขตที่ไม่รวมในเวอร์ชันแรก

ยังไม่ทำระบบสมาชิกออนไลน์ การซิงก์ข้ามอุปกรณ์ การส่งอีเมล การแจ้งเตือนผ่านเซิร์ฟเวอร์ หรือการเรียก AI API จากเว็บโดยตรง เพราะสิ่งเหล่านี้ต้องมีบริการและการตั้งค่าเพิ่มเติม

3. สถาปัตยกรรมที่แนะนำ

สถาปัตยกรรมระดับแอปพลิเคชัน — ไม่ต้องมี Backend ในระยะแรก

Presentation Layer

React Components · Pages · Widgets · Responsive UI

Application Layer

Dashboard Manager · Search · Settings · Task Services

Domain / Utility Layer

Calculator Engine · Date Utilities · Validators · Formatters

Data Access Layer

Settings Repository · Task Repository · Notes Repository

LocalStorage

การตั้งค่าและข้อมูลขนาดเล็ก

IndexedDB

บันทึกและข้อมูลงานที่มีโครงสร้าง

เทคโนโลยี

ส่วน	**ทางเลือก**
Frontend	React + TypeScript
Build tool	Vite
Styling	CSS Modules หรือ CSS variables
Icons	Lucide
Unit testing	Vitest
Component testing	React Testing Library
End-to-end testing	Playwright
Code quality	ESLint + TypeScript
Version control	Git
CI/CD	GitHub Actions

Hosting	GitHub Pages

ไม่จำเป็นต้องติดตั้งไลบรารีทั้งหมดตั้งแต่วันแรก ให้ AI เพิ่มเฉพาะ dependency ที่จำเป็น พร้อมอธิบายเหตุผลก่อนติดตั้ง

3.1 แนวทางจัดเก็บข้อมูล

- LocalStorage: ธีม ภาษา รูปแบบเวลา และค่าตั้งค่าขนาดเล็ก
- IndexedDB: บันทึก งาน รายการโปรด และข้อมูลวิดเจ็ตที่มีโครงสร้าง
- JSON Export: ส่งออกข้อมูลที่ต้องการสำรอง
- Import Validation: ตรวจสอบชนิดข้อมูล ขนาดไฟล์ และเวอร์ชันก่อนนำเข้า
- Schema Version: มีเวอร์ชันข้อมูลเพื่อรองรับการเปลี่ยนโครงสร้างในอนาคต

ข้อควรระวัง: ข้อมูลในเบราว์เซอร์อาจถูกล้างหรือสูญหายได้ และไม่ได้ซิงก์ข้ามอุปกรณ์โดยอัตโนมัติ จึงต้องมีระบบส่งออกและนำเข้าที่ทดสอบแล้ว

4. โครงสร้างไฟล์โปรเจกต์

กำหนดชื่อโปรเจกต์ตัวอย่างว่า productivity-hub

productivity-hub/

public/

ไฟล์สาธารณะและไอคอน

public/icons/

ไอคอนสำหรับ PWA

src/app/

ตัวเริ่มแอปและการตั้งค่าเส้นทาง

src/components/

ส่วนประกอบ UI ที่ใช้ซ้ำ

src/components/layout/

Sidebar, Header, Navigation

src/features/dashboard/

วิดเจ็ตและระบบจัดวาง Dashboard

src/features/clock/

นาฬิกาและเวลา

src/features/calculator/

เครื่องคิดเลข

src/features/timer/

จับเวลาและนับถอยหลัง

src/features/calendar/

ปฏิทินและคำนวณวัน

src/features/tasks/

To-do List

src/features/notes/

ระบบบันทึก

src/features/dev-tools/

เครื่องมือสำหรับนักพัฒนา

src/features/settings/

ธีม ภาษา และการตั้งค่า

src/lib/

ฟังก์ชันช่วยเหลือทั่วไป

src/data/

Repository และการจัดเก็บข้อมูล

src/hooks/

React hooks ที่ใช้ร่วมกัน

src/styles/

Design tokens และ global styles

src/types/

TypeScript types

src/test/

ตัวช่วยสำหรับทดสอบ

docs/

ข้อกำหนด แผนงาน และบันทึกการตัดสินใจ

tests/e2e/

การทดสอบ End-to-end

.github/workflows/

CI และ Deploy

AGENTS.md

กฎกลางสำหรับ AI Agent

README.md

คู่มือโปรเจกต์

CHANGELOG.md

ประวัติการเปลี่ยนแปลง

package.json

คำสั่งและ dependencies

vite.config.ts

การตั้งค่า Vite และ base path

tsconfig.json

การตั้งค่า TypeScript

.gitignore

ไฟล์ที่ไม่ต้องเก็บใน Git

โครงสร้างนี้เป็นเป้าหมาย ไม่จำเป็นต้องสร้างทุกโฟลเดอร์ทันที ให้เพิ่มเมื่อเริ่มพัฒนาโมดูลที่เกี่ยวข้อง

กฎการออกแบบโมดูล

เครื่องมือแต่ละตัวควรรับผิดชอบตัวเอง เช่น calculator ไม่ควรแก้ไขข้อมูล notes โดยตรง หากต้องใช้ข้อมูลร่วมกัน ให้ผ่าน interface หรือ service ที่กำหนดไว้

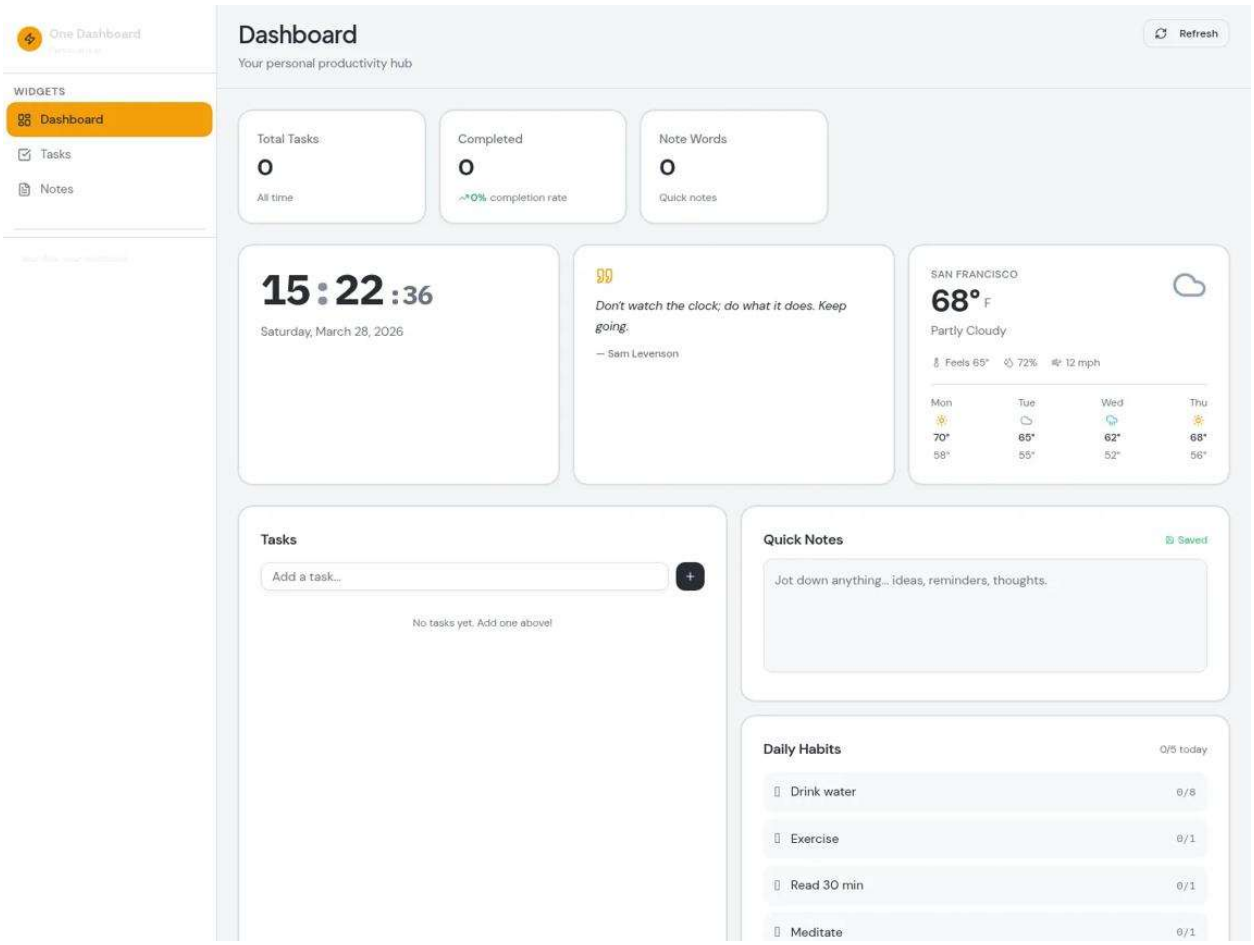
ทุกวิดเจ็ตควรมีข้อมูล metadata อย่างน้อยดังนี้:

```
export interface WidgetDefinition {
  id: string;
  title: string;
  category: string;
  defaultSize: "small" | "medium" | "large";
  supportsResize: boolean;
  supportsOffline: boolean;
}
```

ควรใช้ id ที่คงที่เพื่อให้ระบบจำการจัดวางของผู้ใช้ได้ แม้มีการเปลี่ยนชื่อวิดเจ็ตในภายหลัง

5. ออกแบบ Customizable Dashboard

นี่คือฟีเจอร์ที่ควรเป็นจุดเด่นของโปรเจกต์



blink.new

ความสามารถที่ควรมี

1. เพิ่มและนำวิดเจ็ตออกจากหน้า Dashboard ได้
2. ลากวิดเจ็ตเพื่อเปลี่ยนตำแหน่ง
3. ปรับขนาดวิดเจ็ตได้ตามชนิดที่รองรับ
4. บันทึก Layout อัตโนมัติ
5. ซ่อนวิดเจ็ตที่ไม่ได้ใช้
6. คืนค่า Layout เริ่มต้นได้
7. ปรับแต่งจำนวนคอลัมน์ให้เหมาะกับขนาดหน้าจอ

8. มีโหมดแก้ไข Dashboard แยกจากโหมดใช้งานปกติ

กฎป้องกันปัญหา

- ห้ามให้วิดเจ็ตที่มีข้อผิดพลาดทำให้ Dashboard ทั้งหน้าพัง
- ห้ามบันทึก Layout ที่มี widgetId ซึ่งไม่มีอยู่ในระบบแล้ว
- ต้องมีค่าเริ่มต้นหากข้อมูล Layout เสียหาย
- การย้ายวิดเจ็ตต้องใช้ได้ด้วยคีย์บอร์ด ไม่พึ่ง Drag-and-drop อย่างเดียว
- มือถือควรเปลี่ยนเป็นหนึ่งคอลัมน์หรือรูปแบบที่อ่านง่ายโดยอัตโนมัติ

Design system

กำหนด design tokens เพื่อให้ AI เปลี่ยนหน้าตาทั้งเว็บได้อย่างสม่ำเสมอ

Token	**ค่าเริ่มต้นที่เสนอ**
Primary	#2563EB
Background	#F8FAFC
Surface	#FFFFFF
Text	#0F172A
Border radius	12–16px
Spacing	ระบบระยะห่าง 4px
Font	ฟอนต์ระบบที่รองรับภาษาไทย
Theme	Light / Dark / System

ค่าเหล่านี้เป็นข้อเสนอเริ่มต้น ปรับเปลี่ยนได้ผ่าน design tokens โดยไม่ต้องแก้ CSS ของทุกวิดเจ็ตแยกกัน

6. ระบบ AI ช่วยพัฒนา

ผมแนะนำให้ใช้ AI แบบ Hybrid Development คือมี AI ตัวหลักทำหน้าที่วางแผนและตรวจงาน และ AI เฉพาะด้านช่วยทำงานแต่ละประเภท โดยทุกตัวอ้างอิงกฎกลางชุดเดียวกัน

Orchestrator / AI ผู้ควบคุมกลาง

อ่านข้อกำหนด · แบ่งงาน · ตรวจผล · อัปเดตสถานะ

UI Agent

ดีไซน์และ Responsive

Feature Agent

ฟังก์ชันและโมดูล

Data Agent

Storage และ Migration

QA / Security Agent

ทดสอบและตรวจความเสี่ยง

Human Approval

ผู้ใช้ตรวจสอบงานเสี่ยง อนุมัติ Release และ Deploy

การแบ่ง Agent นี้เป็นการแบ่งบทบาทเชิงกระบวนการ ไม่จำเป็นต้องสร้าง AI Agent อัตโนมัติแยกหลายตัวตั้งแต่แรก คุณสามารถใช้ AI หลายเครื่องมือหรือหลายแชตสลับกันได้ โดยให้ทุกตัวทำตาม AGENTS.md และแผนงานเดียวกัน

ระดับสิทธิ์ AI

ระดับ	**ตัวอย่าง**	**วิธีดำเนินการ**
L1 — งานทั่วไป	เขียนเอกสาร เพิ่ม UI หรือแก้ข้อความ	AI แก้ไฟล์ได้ในขอบเขตที่กำหนด
L2 — งานที่กระทบหลายโมดูล	เปลี่ยน interface หรือ data schema	เสนอแผนก่อน แล้วตรวจ diff และทดสอบ

L3 — งานเสี่ยงสูง	ลบข้อมูล แก้ CI/CD เปลี่ยน dependency สำคัญ	ต้องขออนุมัติก่อนดำเนินการ
L4 — Production	เผยแพร่จริง เปลี่ยนสิทธิ์หรือ Secret	ผู้ใช้อนุมัติขั้นสุดท้าย

แม้ AI ได้รับอนุญาตให้แก้ไฟล์โดยตรง ก็ควรทำงานใน branch แยกหรือ checkpoint ที่ย้อนกลับได้ ห้ามให้ AI commit หรือ push งานเสี่ยงสูงโดยไม่ได้รับอนุมัติ

7. กฎกลางสำหรับ AI: AGENTS.md

ไฟล์นี้เป็นเอกสารสำคัญที่สุดชิ้นหนึ่ง ควรเก็บไว้ที่ root ของ repository

AGENTS.md — Productivity Hub

1. Project Mission

Build a reliable, responsive, customizable All-in-One Productivity Hub.
The application must run as a static web application and be deployable to GitHub Pages.

2. Authoritative Documents

Before changing code, read:

- README.md
- docs/REQUIREMENTS.md
- docs/ARCHITECTURE.md
- docs/ROADMAP.md
- docs/PROJECT_STATUS.md
- Relevant module specifications and ADRs

If documents conflict, stop and report the conflict instead of guessing.

3. Core Technical Rules

- Use React and TypeScript with Vite.
- Keep features modular and loosely coupled.
- Use strict TypeScript settings.
- Prefer small, testable functions and reusable components.

- Keep dependencies minimal.
- Use local-first storage.
- Never assume a backend, API, database service, or secret is configured.
- All deployment paths must support the actual GitHub repository base path.

4. Coding Rules

- Inspect existing code before editing.
- Reuse existing components and conventions.
- Do not overwrite unrelated user changes.
- Do not remove working functionality without approval.
- Avoid any; document justified exceptions.
- Validate imported data and external input.
- Handle loading, empty, success, and error states.
- Do not store sensitive information in logs.
- Do not use dangerouslySetInnerHTML without a reviewed and justified need.

5. Local Data and Privacy

- Store settings in LocalStorage only when appropriate.
- Use IndexedDB for structured user content when appropriate.
- Provide validated import and export.
- Do not transmit user data to external services without explicit approval.
- Do not put API keys, tokens, or secrets in client-side source code.
- Handle storage failures and invalid persisted data safely.

6. Testing Requirements

For each change:

1. Run relevant unit tests.
2. Run lint and type checks.
3. Run the production build.

4. Add or update tests for changed behavior.
5. Report commands executed and actual results.
6. Never claim tests passed unless they were executed successfully.

7. Risk and Approval

L1: Routine UI, documentation, isolated low-risk fixes.
May edit within the assigned scope.

L2: Shared interfaces, storage schemas, dependencies, cross-module changes.
Present a plan and review affected interfaces before implementation.

L3: Data deletion, security-sensitive changes, workflow permissions,
dependency upgrades with broad impact, destructive commands.
Require explicit user approval before execution.

L4: Production deployment, secret rotation, repository permission changes.
Require explicit user approval.

Never expose secrets or run destructive commands without authorization.

8. Git Rules

- Inspect git status before editing.
- Preserve unrelated changes.
- Use feature branches or checkpoints for substantial work.
- Do not force-push.
- Do not commit generated secrets, .env files, or local user data.
- Request approval before committing or pushing high-risk changes.

9. Task Completion Report

After every task, report:

- Objective and scope
- Files created, modified, or deleted
- Important design decisions
- Tests and commands actually executed

- Failures and unresolved issues
- Security and regression risks
- Suggested next task

Update docs/PROJECT_STATUS.md when project status, decisions, milestones, or known issues materially change.

10. Definition of Done

A task is complete only when its acceptance criteria are met, the relevant tests have been run, no known regression is ignored, documentation is updated when needed, and remaining risks are reported.

เอกสารควบคุมงานเพิ่มเติม

ควรสร้างไฟล์ต่อไปนี้ตั้งแต่เริ่มโปรเจกต์:

- docs/PROJECT_STATUS.md — สถานะปัจจุบัน งานเสร็จแล้ว งานค้าง และปัญหาที่ทราบ
- docs/ROADMAP.md — ระยะพัฒนา เป้าหมาย และเกณฑ์ผ่าน
- docs/DECISIONS.md — สรุปการตัดสินใจทางสถาปัตยกรรม
- docs/CHANGELOG.md — บันทึกการเปลี่ยนแปลง
- docs/TEST_PLAN.md — รายการทดสอบและผลที่คาดหวัง
- docs/SECURITY.md — ขอบเขตความปลอดภัยและแนวทางรับมือปัญหา

ควรให้ AI อัปเดตสถานะหลังทำงานแต่ละชุด ไม่ใช่รอจนพัฒนาเสร็จทั้งหมด

8. Prompt กลางสำหรับ AI ผู้ควบคุมโปรเจกต์

ใช้ Prompt นี้กับ AI ที่คุณเลือกให้เป็นผู้ควบคุมกลาง ก่อนเริ่มสั่งงาน Agent เฉพาะด้าน

MASTER PROMPT — Productivity Hub Orchestrator

คุณคือ Senior Software Architect, Technical Lead และ AI Development Orchestrator สำหรับโปรเจกต์ All-in-One Productivity Hub

Project Context

- Project: Productivity Hub
- Editor: Visual Studio Code
- Framework: React + TypeScript + Vite
- Hosting: GitHub Pages
- Data strategy: Local-first
- Version control: Local Git first
- Design: Customizable Dashboard
- Development style: AI-assisted, modular, test-driven where practical

Main Objective

สร้างเว็บรวมเครื่องมืออำนวยความสะดวกที่ใช้งานได้จริง มีคุณภาพสูง

รองรับ Desktop, Tablet และ Mobile พร้อมรองรับการขยายระบบในอนาคต

Your Responsibilities

1. Read and respect AGENTS.md and all authoritative project documents.
2. Inspect the repository and Git status before planning changes.
3. Break work into small, independently verifiable tasks.
4. Define acceptance criteria before implementation.
5. Assign each task to an appropriate specialist role.
6. Identify shared-file conflicts and cross-module dependencies.
7. Review implementation against requirements.
8. Require tests and a successful production build before release.
9. Update project status and record important decisions.
10. Stop and request approval for high-risk actions.

Required Workflow

Phase A: Discovery

- Inspect files, dependencies, existing implementation, and test coverage.
- Identify missing requirements and technical risks.

Phase B: Planning

- Define task ID, scope, expected files, dependencies, and acceptance criteria.
- Classify risk as L1, L2, L3, or L4.
- Avoid overlapping assignments to shared files.

Phase C: Implementation

- Implement only the approved task scope.
- Preserve unrelated changes.
- Use existing conventions and interfaces.
- Add tests for changed behavior.

Phase D: Verification

- Run the relevant tests, lint, type checking, and production build.
- Report real command output and unresolved failures.
- Never fabricate test results.

Phase E: Integration

- Review the diff, check regressions, and verify integration.
- Update documentation and project status.

Phase F: Release

- Confirm all release criteria.
- Obtain explicit user approval before production deployment.

Specialist Roles

- UI Agent: design system, layout, responsive behavior, accessibility.
- Feature Agent: isolated productivity tools and business logic.
- Data Agent: local persistence, validation, migrations, import/export.
- QA Agent: test planning, regression testing, acceptance verification.
- Security Agent: privacy, dependency review, unsafe data handling, deployment risks.

- Release Agent: build pipeline, GitHub Pages configuration, release checklist.

These roles may be fulfilled by separate AI tools or by separate tasks in one AI tool.

Task Management

Use docs/PROJECT_STATUS.md as the source of truth for current progress.

Every task must contain:

- Task ID
- Objective
- Scope and non-goals
- Dependencies
- Risk level
- Acceptance criteria
- Assigned role
- Files expected to change
- Tests required
- Status
- Results and unresolved issues

Allowed statuses:
BACKLOG, READY, IN_PROGRESS, BLOCKED, REVIEW,
TESTING, DONE, and REJECTED.

Do not mark a task DONE until its acceptance criteria are verified.

Final Response Format

Report:

1. Summary
2. Files changed
3. Decisions made
4. Tests executed and results
5. Security and regression findings

6. Outstanding work
7. Recommended next task

Do not claim to have edited files, run commands, or deployed the site
unless those actions were actually performed.

Begin by inspecting the current repository and producing a baseline assessment.
Do not generate the entire application in one uncontrolled operation.

9. Prompt เฉพาะสำหรับ Agent แต่ละฝ่าย

ใช้ Prompt เหล่านี้เมื่อต้องการให้ AI ทำงานเฉพาะด้าน โดยแนบ Task ID และ Acceptance Criteria จาก docs/PROJECT_STATUS.md ทุกครั้ง

A. UI / UX Agent

You are the UI/UX Engineer for Productivity Hub.

Read AGENTS.md, docs/REQUIREMENTS.md, and the assigned task.

Build a consistent, accessible, responsive interface using React, TypeScript,
and the existing design system.

Requirements:

- Support Light, Dark, and System themes.
- Build reusable components and widget containers.
- Support customizable dashboard layout and widget management.
- Provide loading, empty, error, and confirmation states where appropriate.
- Support keyboard navigation and visible focus indicators.
- Ensure Thai text and responsive layouts render correctly.
- Do not invent backend services or add unnecessary dependencies.

Before editing, inspect existing components and Git status.
Implement only the assigned scope.
Run relevant tests and report actual results, changed files,
accessibility concerns, and unresolved issues.

B. Feature Development Agent

You are the Feature Development Engineer for Productivity Hub.

Implement the assigned feature as an isolated, maintainable module.

Rules:

- Follow existing TypeScript types and component conventions.
- Keep calculation and domain logic separate from UI.
- Handle invalid input and edge cases.
- Do not change unrelated modules.
- Do not introduce network calls unless explicitly required and approved.
- Add tests for normal, boundary, and failure cases.
- Do not use placeholder functionality while claiming the feature is complete.

Return a summary of implementation, changed files, acceptance criteria, tests actually executed, and known limitations.

C. Data & Storage Agent

You are the Local Data and Storage Engineer for Productivity Hub.

Design and maintain browser-based persistence using LocalStorage and IndexedDB as appropriate for the data type and expected size.

Requirements:

- Define typed repository interfaces.
- Validate data at import and persistence boundaries.
- Support schema versions and safe migrations.
- Handle unavailable, full, corrupted, or inaccessible storage.
- Implement validated export and import.
- Avoid silent data loss and provide recovery guidance.
- Never transmit local user data externally without explicit approval.
- Do not store secrets or authentication credentials in browser storage.

Before changing a schema, explain migration and compatibility risks.
Test persistence, invalid data, migration, import, export, and failure cases.
Report actual results and unresolved risks.

D. QA Agent

You are the Independent QA Engineer for Productivity Hub.

Do not assume an implementation is correct because the code looks plausible.

Read the requirements and acceptance criteria first.

Verify:

- Functional behavior and edge cases.
- TypeScript, lint, and production build.
- Storage persistence and recovery behavior.
- Responsive layouts and keyboard accessibility.
- Browser refresh and direct URL behavior.
- GitHub Pages base-path compatibility.
- Regression risks and error handling.

Run available tests and report PASS, FAIL, BLOCKED, or NOT TESTED.
Provide reproducible steps for failures.
Do not modify application code during an audit unless explicitly assigned
a separate repair task.
Never claim unexecuted tests passed.

E. Security Agent

You are the Security and Privacy Reviewer for Productivity Hub.

Review the assigned changes for:

- Client-side secret exposure.
- Unsafe HTML rendering and injection.
- Malicious or malformed imported files.
- Sensitive data leakage in logs.
- Dependency and supply-chain risks.

- Insecure external links and network requests.
- Data deletion and migration risks.
- Excessive permissions in CI workflows.
- Misleading privacy and offline claims.

Treat imported content and external data as untrusted.
Recommend mitigations with severity and reproducible evidence.
Do not run destructive commands, rotate secrets, change permissions,
or deploy production without explicit user approval.

Distinguish verified vulnerabilities from theoretical concerns.
Report findings, affected files, severity, recommended fix, and retest results.

F. Release / Deployment Agent

You are the Release Engineer for Productivity Hub.

Prepare a reproducible GitHub Pages deployment.

Verify:

- The intended repository and production branch.
- The Vite base path for the actual deployment URL.
- Lockfile and deterministic dependency installation.
- Successful tests and production build.
- GitHub Actions permissions and deployment configuration.
- Correct asset paths and direct navigation behavior.
- No secrets, local databases, or private files enter the build artifact.
- The final published URL and release evidence.

Do not push, change repository permissions, or deploy production without
explicit user approval.

Report the configuration, commands actually executed, CI results,
known limitations, and rollback procedure.

กฎสำคัญ: ให้ AI ผู้ควบคุมกลางเป็นผู้รวมงานและตัดสินใจเรื่อง interface ส่วน Agent เฉพาะด้านไม่ควรแก้ไฟล์กลางเดียวกันพร้อมกันโดยไม่ประสานงาน

10. แผนพัฒนาเป็นระยะ (Roadmap)

แนะนำให้แบ่งการพัฒนาเป็น 6 ระยะ โดยแต่ละระยะต้องผ่านเกณฑ์รับงานก่อนเริ่มระยะถัดไป

Phase 0

วางระบบและเตรียมโปรเจกต์

เป้าหมาย: Repository พร้อมพัฒนา

- สร้างโครงสร้างไฟล์และเอกสารควบคุม AI
- ตั้งค่า React, TypeScript, Vite, ESLint และ Vitest
- สร้าง Git baseline และตรวจสอบ build
- กำหนด design tokens และรูปแบบการทดสอบ

เกณฑ์ผ่าน: ติดตั้ง dependency, รัน development server และ build ได้

Phase 1

Core Dashboard

เป้าหมาย: ใช้งานหน้าแรกได้จริง

- Layout, Sidebar, Search และ Favorites
- ธีมสว่างและมืด
- นาฬิกา เครื่องคิดเลข และ Timer
- Responsive navigation

เกณฑ์ผ่าน: เครื่องมือหลักใช้งานได้และผ่าน unit tests

Phase 2

Local Data และ Customization

เป้าหมาย: จำการตั้งค่าและข้อมูลได้

- Dashboard drag, resize และซ่อนวิดเจ็ต
- LocalStorage และ IndexedDB
- Notes และ To-do List
- Import / Export พร้อมตรวจสอบข้อมูล

เกณฑ์ผ่าน: Refresh แล้วข้อมูลยังอยู่ และมีวิธีสำรองข้อมูลที่ผ่านการทดสอบ

Phase 3

เครื่องมือเพิ่มเติม

เป้าหมาย: ครอบคลุมงานประจำวัน

- ปฏิทินและคำนวณวันที่
- แปลงหน่วยและเปอร์เซ็นต์
- JSON Formatter, Markdown และ UUID
- ค้นหาเครื่องมือจากหมวดหมู่

เกณฑ์ผ่าน: ทุกเครื่องมือมี validation และ test cases ตามความเสี่ยง

Phase 4

PWA และ Offline

เป้าหมาย: ใช้งานได้โดยไม่พึ่งเครือข่ายในส่วนที่รองรับ

- Web app manifest
- Service worker และ cache strategy
- Offline fallback
- ตรวจสอบการอัปเดตและการกู้คืนหลัง cache เก่า

เกณฑ์ผ่าน: ทดสอบติดตั้ง โหลดซ้ำ และใช้งานออฟไลน์จริงบนเบราว์เซอร์เป้าหมาย

Phase 5

QA, Security และ Release

เป้าหมาย: เผยแพร่จริงอย่างปลอดภัย

- Regression และ end-to-end tests
- Accessibility review
- Dependency audit
- GitHub Actions และ GitHub Pages
- ตรวจสอบ URL จริงและแผน rollback

เกณฑ์ผ่าน: build, tests, deployment และ privacy checks ผ่านก่อนอนุมัติเผยแพร่

ระยะเหล่านี้เป็นลำดับความสำคัญ ไม่ใช่การรับประกันระยะเวลาหรือการรับรองว่าโปรเจกต์จะผ่านเกณฑ์โดยอัตโนมัติ

11. วิธีทดสอบระบบก่อนใช้งานจริง

กำหนดไฟล์ docs/TEST_PLAN.md แล้วแบ่งการทดสอบเป็น 5 กลุ่ม

11.1 Test matrix

กลุ่มทดสอบ	สิ่งที่ต้องตรวจสอบ	เกณฑ์ผ่าน
Unit	การคำนวณ วันเวลา การตรวจสอบข้อมูล	ผลลัพธ์ถูกต้องตามกรณีทดสอบ
Integration	Dashboard เชื่อมกับ Storage	เพิ่มและโหลดข้อมูลกลับได้
E2E	ขั้นตอนใช้งานตั้งแต่เปิดเว็บถึงบันทึกงาน	Workflow หลักทำงานครบ
Responsive	Desktop, Tablet, Mobile	ไม่มีส่วนสำคัญล้นหรือใช้งานไม่ได้
Security / Privacy	Import, storage, external requests	ไม่พบปัญหาที่ไม่ผ่านเกณฑ์ความเสี่ยงที่กำหนด

11.2 กรณีทดสอบที่ห้ามข้าม

Checklist สำหรับ QA

0/20 ข้อ

Dashboard

เพิ่มวิดเจ็ต แล้วย้ายตำแหน่งได้

เปลี่ยนขนาดและซ่อนวิดเจ็ตได้

รีเฟรชแล้ว Layout ยังอยู่

ข้อมูล Layout ผิดรูปแบบแล้วระบบกลับสู่ค่าเริ่มต้นได้

Calculator และ Timer

คำนวณเลขติดลบ ทศนิยม และเปอร์เซ็นต์

ป้องกันการหารด้วยศูนย์และค่าที่ไม่ถูกต้อง

จับเวลา หยุด และเริ่มใหม่ได้

ทดสอบกรณีแท็บอยู่เบื้องหลังเป็นเวลานาน

Local Data

สร้าง แก้ไข และลบงานได้

ส่งออกและนำเข้าข้อมูลกลับได้

ปฏิเสธไฟล์นำเข้าที่ผิดรูปแบบ

แสดงข้อผิดพลาดเมื่อพื้นที่เก็บข้อมูลไม่พร้อม

Deployment

เปิดหน้าแรกจาก URL จริงได้

ไฟล์ CSS, JavaScript และไอคอนโหลดได้

รีเฟรชและเปิด URL ที่รองรับโดยตรงได้

เปิดเว็บหลังมีการอัปเดตเวอร์ชันได้

Accessibility

ใช้ฟังก์ชันหลักด้วยคีย์บอร์ดได้

Focus indicator มองเห็นได้

ชื่อปุ่มและช่องกรอกมีความหมาย

ข้อความไทยอ่านได้บนมือถือ

ล้างเครื่องหมายคัดลอกผล Checklist

Checklist นี้ใช้ติดตามการตรวจสอบ ไม่ได้เป็นหลักฐานว่าระบบผ่านการทดสอบแล้ว ต้องดำเนินการทดสอบจริงและบันทึกผลไว้ด้วย

11.3 คำสั่งทดสอบมาตรฐาน

กำหนด scripts ใน package.json ให้มีคำสั่งมาตรฐานอย่างน้อยดังนี้

```
{
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview",
    "lint": "eslint .",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:e2e": "playwright test"
  }
}
```

นี่คือตัวอย่าง scripts ที่ต้องปรับให้ตรงกับไฟล์ตั้งค่าและเวอร์ชัน dependency จริงของโปรเจกต์ ไม่ควรนำไปแทนที่ package.json เดิมโดยไม่ตรวจสอบ

เมื่อกำหนด scripts และติดตั้งเครื่องมือทดสอบครบแล้ว ให้รัน:

npm run lint

npm run test

npm run build

npm run test:e2e

หากยังไม่ได้ติดตั้ง Playwright หรือ browser dependencies คำสั่ง E2E อาจยังรันไม่ได้ ต้องติดตั้งและตั้งค่าให้เสร็จก่อน

12. ความปลอดภัยและความเป็นส่วนตัว

แม้เว็บจะไม่มี Backend ก็ยังมีความเสี่ยงจากการประมวลผลข้อมูลในเบราว์เซอร์ การนำเข้าไฟล์ และ dependencies

12.1 สิ่งที่ต้องป้องกัน

- ห้ามใส่ API Key, token หรือ password ใน source code ที่เผยแพร่สู่สาธารณะ
- ห้ามถือว่าข้อมูลใน LocalStorage หรือ IndexedDB ถูกเข้ารหัสหรือปลอดภัยจากสคริปต์ที่เป็นอันตรายโดยอัตโนมัติ
- ตรวจสอบชนิดและขนาดไฟล์ก่อนนำเข้าข้อมูล
- ไม่รัน JavaScript ที่มากับไฟล์นำเข้า
- หลีกเลี่ยงการแสดง HTML จากผู้ใช้โดยตรง
- ใช้ dependency เท่าที่จำเป็นและตรวจสอบช่องโหว่เป็นระยะ
- จำกัด GitHub Actions permissions เท่าที่ workflow ต้องใช้
- ไม่เก็บข้อมูลส่วนตัวของผู้ใช้ไว้ใน Git repository

12.2 นโยบายข้อมูลที่แนะนำ

ข้อมูล	วิธีจัดเก็บ
ธีมและ Layout	LocalStorage
รายการงานและบันทึก	IndexedDB
API Secret	ไม่เก็บในเว็บ Static
Backup	ไฟล์ที่ผู้ใช้ดาวน์โหลดเอง
Analytics	ไม่ติดตั้งในเวอร์ชันแรก
External API	ปิดไว้จนกว่าจะมีการกำหนดและอนุมัติ

ข้อจำกัดที่ต้องเข้าใจ: Local-first ไม่ได้หมายความว่าข้อมูลจะปลอดภัยจากทุกกรณี หากผู้ใช้เปิดเว็บบนอุปกรณ์สาธารณะ มีส่วนขยายเบราว์เซอร์ที่ไม่ปลอดภัย หรือดาวน์โหลดไฟล์สำรองไปไว้ในที่เปิดเผย ข้อมูลก็อาจรั่วไหลได้

12.3 นโยบายการอัปเดต

ก่อนเปลี่ยนโครงสร้างข้อมูลที่บันทึกอยู่ ให้ AI ทำตามลำดับนี้:

1. ระบุ schema เดิมและ schema ใหม่
2. สร้าง migration ที่รองรับข้อมูลเดิม
3. สำรองข้อมูลก่อนการเปลี่ยนแปลงที่เสี่ยง
4. ทดสอบกับข้อมูลตัวอย่างและข้อมูลผิดรูปแบบ
5. มีแนวทางกู้คืนเมื่อ migration ล้มเหลว
6. ขออนุมัติหากการเปลี่ยนแปลงอาจทำให้ข้อมูลจริงสูญหาย

13. วิธี Deploy ขึ้น GitHub Pages จริง

สำหรับโปรเจกต์ React + Vite ควรใช้ GitHub Actions สร้างไฟล์เว็บแล้วเผยแพร่จากโฟลเดอร์ dist ซึ่งเป็นแนวทางที่เอกสาร Vite แนะนำสำหรับ GitHub Pages



Vite

+1

ขั้นตอนที่ 1 — สร้างโปรเจกต์ใน VS Code

หากเริ่มโปรเจกต์ใหม่ ให้เปิด Terminal ใน VS Code แล้วรัน:

```
npm create vite@latest productivity-hub -- --template react-ts
cd productivity-hub
npm install
npm run dev
```

ถ้าต้องการทดสอบการทำงานของโปรเจกต์ ให้เปิด URL ที่ Vite แสดงใน Terminal

หาก PowerShell ใน Windows บล็อก npm.ps1 ให้ลองใช้ Command Prompt ใน VS Code หรือเรียก npm.cmd แทนการเปลี่ยน Execution Policy ของระบบทันที

ขั้นตอนที่ 2 — ตั้งค่า Git

```
git init
git add .
git commit -m "chore: initialize productivity hub"
```

ก่อน git add . ให้ตรวจ .gitignore และ git status เพื่อไม่ให้มีไฟล์ส่วนตัวหรือไฟล์ลับติดเข้าไป

สร้าง Repository บน GitHub แล้วเชื่อม remote ตาม URL จริงที่ GitHub ให้มา:

```
git branch -M main
git remote add origin <YOUR_REPOSITORY_URL>
git push -u origin main
```

แทน <YOUR_REPOSITORY_URL> ด้วย URL ของ repository ของคุณจริง ๆ

ขั้นตอนที่ 3 — ตั้งค่า Base Path

หาก Repository ชื่อ productivity-hub และ URL เป็น https://USERNAME.github.io/productivity-hub/ ต้องตั้งค่า base ให้ตรงกับชื่อ Repository



Vite

ใน vite.config.ts:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  base: "/productivity-hub/",
});
```

ตัวอย่างนี้ใช้ได้เมื่อชื่อ Repository เป็น productivity-hub จริง หากใช้ Repository แบบ USERNAME.github.io หรือ Custom Domain ที่ root ให้ใช้ base: "/" แทน

ขั้นตอนที่ 4 — สร้าง Workflow สำหรับ Deploy

สร้างไฟล์ .github/workflows/deploy.yml

```
name: Deploy Productivity Hub

on:
  push:
    branches: [main]
  workflow_dispatch:
```

```
permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: github-pages
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci
```

```
    - name: Lint
      run: npm run lint

    - name: Test
      run: npm run test

    - name: Build
      run: npm run build

    - name: Setup Pages
      uses: actions/configure-pages@v5

    - name: Upload artifact
      uses: actions/upload-pages-artifact@v3
      with:
        path: dist

    - name: Deploy
      uses: actions/deploy-pages@v4
```

ตัวอย่างนี้แสดงโครงสร้าง workflow เพื่อเริ่มต้นใช้งานจริง แต่ควรตรวจเวอร์ชัน Actions ที่รองรับและอัปเดตเป็นเวอร์ชันที่เหมาะสมก่อนใช้ Production โดยเฉพาะการ pin เวอร์ชันหรือ commit SHA ตามนโยบายความปลอดภัยของโปรเจกต์ เอกสาร Vite มีตัวอย่าง workflow ที่ใหม่กว่าให้ใช้เป็นข้อมูลอ้างอิง



Vite

หากยังไม่ได้ติดตั้งและตั้งค่า npm run lint หรือ npm run test ให้ตรงกับโปรเจกต์ Workflow จะหยุดเมื่อคำสั่งเหล่านั้นล้มเหลว ซึ่งเป็นพฤติกรรมที่ควรรักษาไว้ ไม่ควรลบขั้นตอนตรวจสอบเพื่อให้ Deploy ผ่านโดยไม่แก้ปัญหา

ขั้นตอนที่ 5 — เปิดใช้งาน GitHub Pages

1. เปิด Repository บน GitHub
2. เข้า Settings → Pages
3. เลือก Source เป็น GitHub Actions
4. เปิดแท็บ Actions แล้วตรวจผล Workflow
5. เมื่อ Deploy สำเร็จ เปิด URL ที่ GitHub รายงาน
6. ทดสอบการโหลดไฟล์ การรีเฟรช การบันทึกข้อมูล และการใช้งานบนมือถือ

ขั้นตอนนี้เป็นการเผยแพร่เว็บจริง แต่ยังไม่ถือว่าโปรเจกต์พร้อมใช้งานจนกว่าจะผ่านการทดสอบและตรวจสอบ URL ที่เผยแพร่แล้ว



GitHub Docs

+1

ข้อควรระวังสำหรับ PWA

การติดตั้ง Service Worker ต้องใช้ Secure Context โดยทั่วไปคือ HTTPS หรือ localhost และต้องออกแบบ cache ให้รองรับ Base Path ของ GitHub Pages ด้วย



MDN

อย่าเพิ่งเปิด Cache ทุกไฟล์หรือทำ Offline Mode จนกว่าจะทดสอบกรณีมีเวอร์ชันใหม่ เพราะ Cache เก่าอาจทำให้ผู้ใช้เห็นหน้าเว็บที่ไม่ตรงกับโค้ดล่าสุด

14. Prompt สำหรับให้ AI ลงมือสร้างโปรเจกต์จริง

ใช้ Prompt นี้เป็นงานเริ่มต้นใน VS Code AI Agent หลังจากสร้างโฟลเดอร์โปรเจกต์แล้ว

TASK-001: Bootstrap Productivity Hub

Read AGENTS.md first. If it does not exist, create it using the approved project rules.

Objective

Initialize a production-oriented React + TypeScript + Vite project
for an All-in-One Productivity Hub.

Scope

1. Inspect the repository and current Git status.
2. Identify existing user files and preserve them.
3. Set up or verify React, TypeScript, and Vite.
4. Configure linting and unit testing.
5. Create the initial project documentation.
6. Create a responsive application shell.
7. Implement an initial Dashboard with placeholder widget registration only.
8. Prepare the GitHub Pages base-path configuration.
9. Add a minimal GitHub Actions workflow after checking the available scripts.
10. Provide local development and deployment instructions.

Initial Files

- AGENTS.md
- README.md
- docs/REQUIREMENTS.md
- docs/ARCHITECTURE.md
- docs/ROADMAP.md
- docs/PROJECT_STATUS.md
- docs/TEST_PLAN.md
- docs/SECURITY.md

Constraints

- Do not add a backend or external API.
- Do not add an AI API key or secret.
- Do not create a fake login system.
- Do not claim unfinished widgets are functional.
- Keep dependencies minimal.
- Do not overwrite unrelated files.
- Do not commit, push, or deploy without authorization.
- Use a configurable repository base path.
- Keep the app usable with local data only.

Acceptance Criteria

- The project installs reproducibly from its lockfile.
- The development server starts.
- Type checking, lint, unit tests, and production build pass.
- The Dashboard shell renders.
- The UI is responsive and keyboard accessible.
- Documentation describes the actual implementation.
- The deployment workflow matches the package scripts.
- All executed commands and their results are reported accurately.

Required Report

Return:

1. Initial repository assessment.
2. Proposed file changes.
3. Implementation summary.
4. Commands executed and their real results.
5. Failures or blocked steps.

6. Security risks.
7. Recommended next task.

Begin with repository inspection. Do not implement all productivity features in one operation.

15. วิธีทำงานประจำวันร่วมกับ AI

ใช้วงจรนี้กับทุก Task เพื่อป้องกัน AI แก้โค้ดหลายส่วนพร้อมกันจนควบคุมไม่ได้

1. Plan

ให้ AI อ่านสถานะและเสนอแผนก่อนแก้โค้ด

2. Implement

ทำทีละ Task และจำกัดขอบเขตไฟล์ที่แก้

3. Review

ตรวจ Git diff และให้ QA Agent ตรวจงานแยกจากผู้เขียน

4. Verify

รัน Test, Lint, Type Check และ Build

5. Checkpoint

บันทึก Git commit เมื่อผ่านเกณฑ์และได้รับอนุญาตตามระดับความเสี่ยง

6. Update

อัปเดต PROJECT_STATUS.md แล้วเลือก Task ถัดไป

16. สิ่งที่ควรทำเป็นอันดับแรก

ผมแนะนำให้เริ่มด้วย 3 งานนี้ตามลำดับ:

1. TASK-001 — เตรียมโครงสร้างโปรเจกต์ เอกสารกลาง และระบบทดสอบ
2. TASK-002 — สร้าง Dashboard UI พร้อมระบบ Theme, Search และ Widget Registry
3. TASK-003 — พัฒนานาฬิกา เครื่องคิดเลข และ Timer พร้อม Unit Tests

ยังไม่ควรเริ่มด้วยการสร้างทุกเครื่องมือพร้อมกัน เพราะจะทำให้การตรวจสอบคุณภาพและการหาจุดผิดพลาดยากขึ้น

เป้าหมายแรกคือให้มีเว็บที่รันในเครื่องได้จริง โครงสร้างชัดเจน ผ่านการทดสอบพื้นฐาน และพร้อมเผยแพร่ผ่าน GitHub Pages ก่อน แล้วค่อยเพิ่มฟีเจอร์ทีละโมดูล

หมายเหตุ: แผนและ Prompt ข้างต้นเป็นข้อกำหนดสำหรับนำไปใช้พัฒนา ยังไม่ได้สร้างไฟล์ลงในเครื่องหรือ Repository ของคุณ และยังไม่มีการรัน Test หรือ Deploy จริงในขั้นตอนนี้